

Question 1

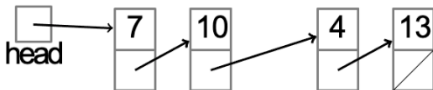
1. What is the primary advantage of using a singly linked list over an array when inserting or deleting elements at arbitrary positions?

Question 2

Consider the diagram shown for a singly linked list where a new node is inserted at the back of the list (PushBack operation). The time complexity of this operation is $O(n)$ when there is no tail pointer, as traversal is required to reach the end of the list.

PushBack(key)

```
node ← new node
node.key ← key
node.next = nil
if tail = nil:
    head ← tail ← node
else:
    tail.next ← node
    tail ← node
```



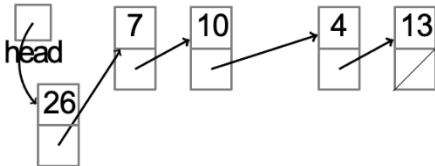
1. Why is the time complexity of the PushBack operation $O(n)$ in the absence of a tail pointer?
2. What modifications could be made to this operation to reduce its time complexity to $O(1)$?
3. Discuss the trade-offs between maintaining a tail pointer and the overhead it introduces in the linked list.

Question 3

Consider the diagram shown for a singly linked list where a new node is inserted at the front of the list (PushFront operation). The time complexity of this operation is $O(1)$.

PushFront(key)

```
node ← new node  
node.key ← key  
node.next ← head  
head ← node  
if tail = nil:  
    tail ← head
```



Explain why the time complexity of the PushFront operation is $O(1)$. What happens to the head pointer after the new node is inserted? Compare the efficiency of adding an element to the front of a singly linked list (PushFront) versus adding an element to the front of an array.

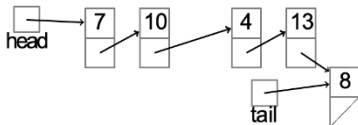
Question 4

Consider the diagram shown for a singly linked list where the last node is removed from the list (PopBack operation). The time complexity of this operation is $O(n)$, even when there is a tail pointer, because traversal is required to locate the second-to-last node.

PopBack()

```
if head = nil:  ERROR: empty list
if head = tail :
    head ← tail ← nil
else:
    p ← head
    while p.next.next ≠ nil:
        p ← p.next
```

PopBack
(with tail)



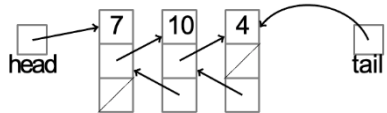
1. Why does the PopBack operation still have a time complexity of $O(n)$ despite the presence of a tail pointer?
2. What optimizations can be made to reduce the overhead of this operation, specifically when updating the tail pointer?
3. Discuss how a doubly linked list would improve the efficiency of the PopBack operation and how that would affect time complexity.

Question 5

Consider the diagram of a doubly linked list where a node is removed from the end of the list (PopBack operation). The time complexity of this operation is $O(1)$ due to the presence of both prev and next pointers, allowing direct access to the second-to-last node.

PopBack()

```
if head = nil: ERROR: empty list
if head = tail :
    head ← tail ← nil
else:
    tail ← tail .prev
    tail .next ← nil
```



1. Why is the time complexity of the PopBack operation in a doubly linked list $O(1)$?

How do the prev and next pointers enhance the efficiency of this operation compared to a singly linked list?

What are the memory trade-offs between maintaining a doubly linked list and a singly linked list?